4 Experiments and Results

This chapter reports the experiments on the GazeCapture data set [1] and on the personal recording sessions. It first describes the model, the data and the evaluation protocol (Section 4.1). It then analyses the training hyperparameters (Section 4.2), the spatial distribution of the prediction error and the loss functions (Section 4.3), and the personalization sessions (Section 4.4). Section 4.7 discusses what these results mean for the rest of the thesis.

All values were re-checked against the experiment logs. During this check an error in the evaluation code was found and corrected (Section 4.1.3). All model errors in this chapter are reported with the corrected metric.

4.1 Experimental Setup

4.1.1 Model

- **Backbone:** ResNet-18 [2], pretrained on ImageNet (`ResNet18_Weights.DEFAULT`).
- **Regression head:** four fully connected layers $512 \to 512 \to 256 \to 128 \to 2$ with ReLU, dropout $p = 0.2$ after the second layer, and an output activation (default: Sigmoid).
- **Optimizer:** AdamW, weight decay 10^{-5}.
- **Default loss:** L1 loss (`nn.L1Loss`), see Section 4.3.3.
- **Early stopping:** patience of three epochs, maximum 500 epochs.

4.1.2 Data and Splits

GazeCapture contains 347 085 labelled frames of 262 subjects in the version used here. Two splits are used:

- `norm_subject`: subject-disjoint split (no person appears in both training and test data);
- `norm_random`: frame-level split, the same person can appear in training and test data.

For the hyperparameter study, 1000 training and 200 test frames are drawn *at random* from the split files (fixed seed 0, `-sample-seed 0`). For `norm_subject` the training subset contains 179 subjects and the test subset 45 subjects, with no overlap. (Earlier runs used the *first* rows of the sorted split files, i.e. only five training subjects and one test subject; these runs are no longer used for the tables.)

4.1.3 Evaluation Metric

All gaze targets are normalized to the unit square $[0, 1]^2$ (Section 4.1.5). The error is the root mean squared *Euclidean* distance between prediction $\hat{\boldsymbol{p}}_i$ and target $\boldsymbol{p}_i$, expressed as a percentage of the screen diagonal $\sqrt{2}$:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

A value of 0 % is a perfect prediction, and 100 % is an error as large as the full screen diagonal.

4.1.4 Protocol

The hyperparameter study is run with the command

```
python exp.py -experiment all -sample-seed 0 -val-size 200.
```

One axis (for example the batch size) is changed, while all other values stay at the default configuration (Table 4.1). Each configuration is trained five times. Early stopping uses a subject-disjoint validation set of 200 frames, and the reported value is the *test* error at the epoch with the lowest validation error, so the test set is not used for model selection. The tables report the mean test error, the two-sided 98 % confidence interval (CI) of the mean (Student-t, four degrees of freedom) and the mean run time (defined below).

What "Run time" measures. The column *Run time* is the wall-clock duration of *one complete training run*, averaged over the five runs. A run starts with the evaluation of the untrained model (epoch 0) and ends when early stopping stops the training; it includes all training epochs and the evaluation on the training, validation and test data after every epoch. It is *not* the time per epoch, and it does *not* include loading the images or computing the cached backbone features, which happens once before the run. Because early stopping ends the runs after a different number of epochs, the time also reflects how long a configuration keeps improving.

Table 4.1: Default configuration of `exp.py`. Each sweep changes exactly one of these values.

Parameter	Value	Parameter	Value
Split	`norm_subject`	Trainable layers	FC only
Train / test size	1000 / 200[a]	Output activation	Sigmoid
Batch size	32	Loss	L1 (`standard`)
Learning rate	10^{-4}	Max. epochs / patience	500 / 3
Weight decay	10^{-5}	Validation set	200 frames[b]
Repeats	5		

[a] drawn at random, seed 0. [b] from one subject that is not in the training or test subset.

Reading the tables. In every hyperparameter table the selected option is marked with †. The column "p vs. †" gives the p-value of a two-sided Mann–Whitney U test between the five runs of that option and the five runs of the selected one. A rank test is used because five runs are too few to check normality and several options contain single outlier runs (e.g. one ReLU run at 46.98 %). With $5 + 5$ runs the smallest possible p-value is 0.008; it means that *every* run of one option is better than *every* run of the other. A mean is printed in **bold** only if the selected option is *significantly better* than every other option of the same table ($p < 0.05$ and lower mean).

Three properties of this protocol must be kept in mind:

1. **Same data in all repeats.** The seed fixes the train, test and validation subsets, so the five repeats differ only in weight initialization, data order and dropout. The CI therefore measures training randomness, not the variation between different data subsets.
2. **Small validation set.** The 200 validation frames come from a single subject. Early stopping is therefore noisy, but the test set is never used for model selection.
3. **Training time.** For FC-only runs the head is trained on cached ResNet features (512 numbers per image instead of the image). One run therefore takes only about one second, and the differences between these times are not relevant. The times of FC + Layer 4 and Full ResNet contain the forward and backward pass through the backbone in every epoch and are therefore much longer; they cannot be compared directly with the FC-only times.

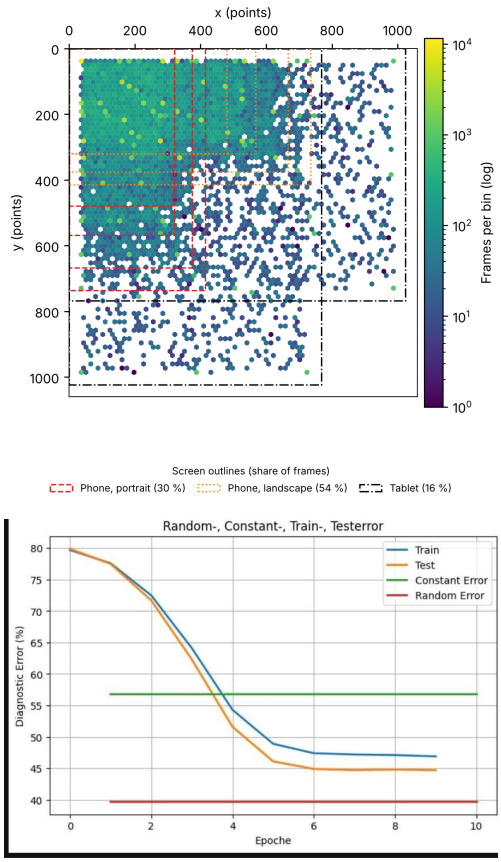
4.1.5 Target Normalization

The raw gaze targets of GazeCapture are given in screen points of the recording device, measured from the upper left corner of the screen. Their range therefore depends on the screen size and the orientation of the device: the 347 085 frames of the 262 subjects come from ten screen formats, from 320×480 to 1024×768 points, recorded in portrait and

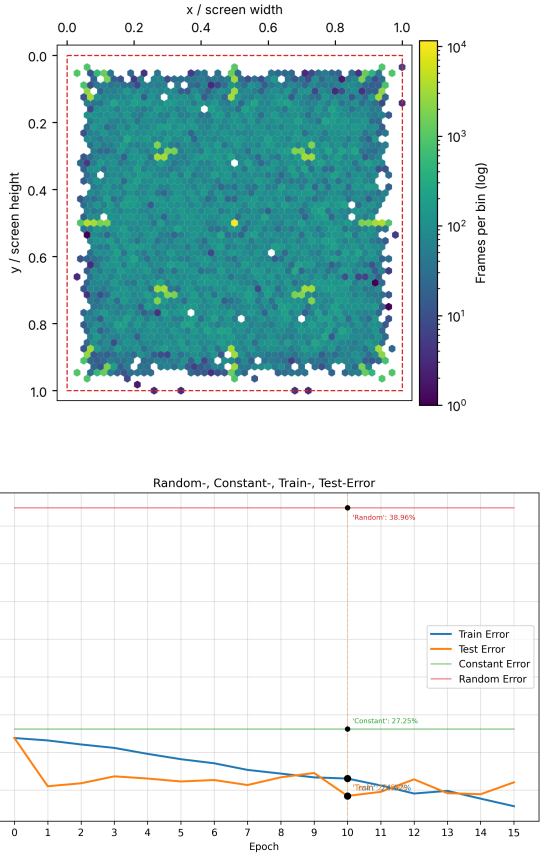
landscape orientation. The targets are therefore normalized with the screen width W and height H of the frame (`main.py`; values outside $[0, 1]$ are clipped, which affects 0.006 % of the frames):

$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

Figure 4.1 shows the effect. Without normalization the targets of the many phone frames are concentrated in the upper left area, and the same coordinate means a different screen position on every device (Figure 4.1a); the error values are therefore not comparable between devices (left column). After normalization all targets lie in $[0, 1]^2$ and Equation (4.1) is well-defined (right column). All further experiments use normalized targets.



(a) Raw targets in screen points (density of all frames, logarithmic colour scale); dashed lines: outlines of the screen formats. Bottom: learning curve of an early run on raw coordinates; the values are not percentages of a common screen diagonal and cannot be compared with the other figures.



(b) Normalized targets (same representation; dashed: unit square). The bright cells are calibration points shown at the same relative positions on all devices. Bottom: learning curve on `norm_random` (1000/200) with the metric of Equation (4.1). Epoch 0 is the untrained model. The horizontal lines are baselines of the *full* data set; see text.

Figure 4.1: Raw versus normalized gaze targets.

Why does the curve start at the constant and not at the random baseline? In Figure 4.1b the curves start at 26.77 % at epoch 0, i.e. *before* the first training step. One could expect the untrained model to behave like the random baseline (about 40 %). This expectation does not hold, for the following reason.

The random baseline guesses a *different*, uniformly distributed point for every image, spread over the whole screen. An untrained network does not do this. Its weights are initialized with small random values, so the last linear layer produces values close to 0 for every image, and the Sigmoid maps them to $\sigma(0) = 0.5$. The untrained model therefore predicts almost the *same* point, the screen centre (0.5, 0.5), for all images. This is exactly what the constant baseline does. Hence the correct expectation is:

- epoch 0 $\approx$ constant baseline of the *same* test data (a "always look at the centre" guess);
- training must bring the curve *below* this value; only then has the model learned something from the images;
- the random baseline is an upper reference for "pure guessing"; a model should never be above it.

The measured values confirm this: on the 200 test images of this run the constant baseline is 26.76 % (training mean) or 26.71 % (screen centre), which equals the start value of 26.77 %. The horizontal line in the old figure (27.25 %) was computed on the *full* data set and therefore lies slightly higher; this made the curve seem to start "below the baseline". The plotting code was corrected: the baselines are now computed on the same train/test subset as the curves (`metrics.subset_baselines`; `Compare_notebook.ipynb`, Section 4, which also re-creates this figure).

4.1.6 Baselines

Two reference predictors are used. The *constant baseline* predicts the mean target of the training data for every test image. The *random baseline* draws predictions uniformly from $[0, 1]^2$. A model is only useful if it is clearly better than the constant baseline.

Table 4.2: Baseline errors E_{diag} in %. Top: full data set (`exp.py`, bootstrap of 30 samples with 2000 frames, mean and 98 % CI). Bottom: the same random train/test subsets (seed 0) that are used for model training (no CI, one fixed subset).

Data	Constant baseline	Random baseline
Full data set (347 085 frames)	27.28 [27.15, 27.40]	39.82 [39.60, 40.04]
`norm_subject`, 1000/200	28.46	40.36
`norm_subject`, 10 000/2000	27.62	39.96
`norm_random`, 1000/200	27.66	39.83

The baselines of the earlier draft (30.99 % and 42.70 %) came from an older notebook run that is not part of the logs; they are replaced by the logged values in Table 4.2. The lower part of the table is the correct reference for all model results in Section 4.2.

4.2 Training Configuration

All results in this section come from one run (`exp_results_all.json`; `exp.py`, corrected metric, random subsets with seed 0, validation-based early stopping, five repeats per configuration). The same-subset constant baseline of the default configuration is 28.46 %.

4.2.1 Batch Size

Table 4.3: Batch size. Mean test error over five runs with 98 % CI and mean duration of one complete training run (FC-only head on cached features, see Section 4.1.4).

Batch size	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
32†	28.39	[28.21, 28.58]	—	0.019
64	28.35	[28.29, 28.41]	0.69	0.012
128	28.38	[28.30, 28.47]	0.84	0.009

Table 4.3 shows that the three batch sizes give the same test error: the means differ by at most 0.05 percentage points, the confidence intervals overlap, and the rank test finds no difference ($p \geq 0.69$). The training times are below one second per run, because only the small head is trained on cached features; the differences in time are therefore not relevant.

A batch size of 32 is kept as default. The experiment shows no disadvantage of this value, and there are two practical arguments for a small batch in the personalization setting: (i) a personal session contains only a few hundred training images, so a small batch gives more parameter updates per epoch, and (ii) the gradient noise of small batches is commonly associated with a mild regularizing effect. The data of this experiment do *not* prove an advantage of 32 over 64 or 128.

4.2.2 Learning Rate

Table 4.4 shows no significant difference between the four learning rates in the five-run sweep: all means lie between 28.33 % and 28.43 %, the confidence intervals overlap, and no rank test is significant ($p \geq 0.056$). The final error alone therefore cannot justify the choice. The choice of $\eta = 10^{-4}$ is based on the *convergence behaviour* in Figure 4.2:

Table 4.4: Learning rate. Mean test error over five runs with 98 % CI and mean duration of one complete training run.

Learning rate	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
10^{-3}	28.43	[28.33, 28.54]	0.15	0.018
$10^{-4\dagger}$	28.37	[28.32, 28.42]	—	0.020
10^{-5}	28.33	[28.28, 28.38]	0.056	0.022
10^{-6}	28.37	[28.21, 28.52]	0.42	0.055

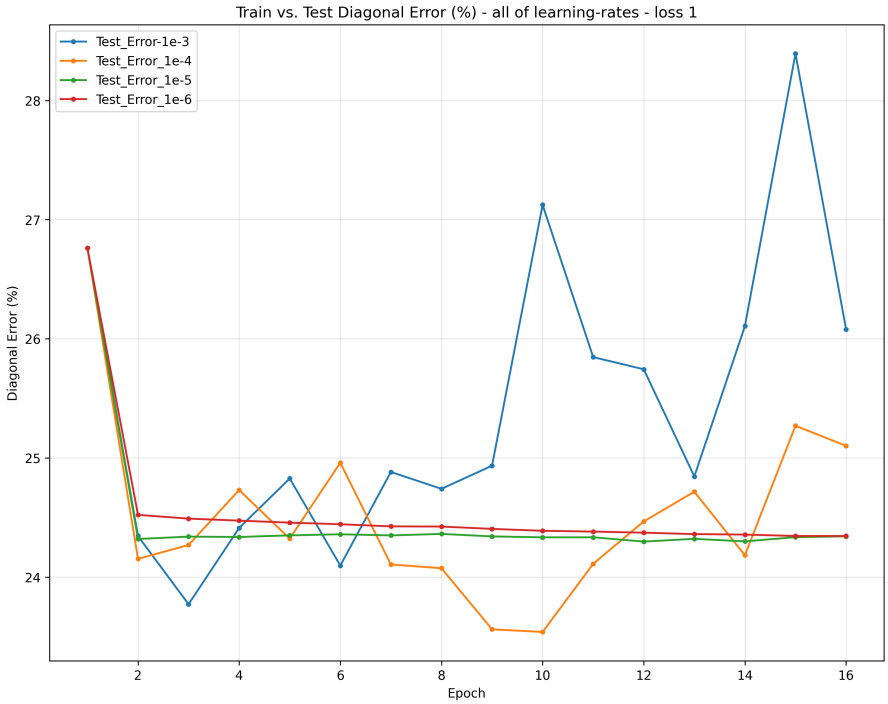


Figure 4.2: Test error curves of single training runs with four learning rates (`norm_random`, 1000/200, 15 epochs without early stopping, metric of Equation (4.1)). The first point (epoch 1 on the axis) is the untrained model.

- $\boldsymbol{\eta = 10^{-3}}$: the steps are too large. After a fast first decrease the test error oscillates strongly and grows again (peaks of about 27.1 % and 28.4 %); the run does not converge to a stable minimum.
- $\boldsymbol{\eta = 10^{-4}}$: reaches the lowest test error of all curves (about 23.6 % around epoch 10). The curve still fluctuates, but much less than with 10^{-3}, and early stopping can keep the best state.
- $\boldsymbol{\eta = 10^{-5}}$ **and** $\boldsymbol{10^{-6}}$: the curves are very smooth and converge monotonically, but they stay on a plateau of about 24.3 %. The step size is too small to leave this region within the training budget; the result is stable but clearly not the best reachable one.

The training time is *not* used as a criterion here: it is below one second for all learning rates (cached features), and the decision depends only on convergence behaviour and the reachable error. Note that Figure 4.2 shows single runs; the five-run sweep in Table 4.4 does not confirm a significant difference.

4.2.3 Trainable Layers

Three fine-tuning depths are compared (Table 4.5): only the regression head (FC only), the head plus the last residual stage (`layer4`), and the complete network.

Table 4.5: Trainable parameters of the three fine-tuning configurations (ResNet-18 with regression head, 11 603 650 parameters in total).

	FC only	FC + Layer 4	Full ResNet
Trainable parameters	427 138	8 820 866	11 603 650
Share of all parameters	3.68 %	76.02 %	100.00 %
Backbone features cacheable	yes	no	no

Table 4.6: Trainable layers. Mean test error over five runs with 98 % CI and mean duration of one complete training run. The FC-only time excludes the backbone forward pass (cached features).

Trainable layers	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
FC only†	28.36	[28.23, 28.49]	—	0.015
FC + Layer 4	29.07	[26.58, 31.55]	0.22	9.160
Full ResNet	29.39	[26.99, 31.78]	0.31	15.366

FC only has the lowest mean error and by far the narrowest interval (Table 4.6). Training more layers does not help; it makes the result *less stable*: one of the five FC + Layer 4 runs (31.72 %) and two of the five full fine-tuning runs (30.70 % and 31.18 %) end clearly above the constant baseline (28.46 %), while all FC-only runs stay between 28.28 % and 28.47 %. With only five runs the differences are not significant ($p = 0.22$ and 0.31), so the FC-only

value is not printed in bold. The training time grows strongly with the number of trainable parameters: 9.2 min per run for FC + Layer 4 and 15.4 min for the full network.

FC only is selected for the personalization experiments for three reasons:

1. **No loss in accuracy, more stability:** in this experiment, training more layers does not reduce the error but increases the spread between runs.
2. **Robustness with few data:** a personal session has only a few hundred images. Updating millions of backbone parameters with so few samples risks overfitting and distorts the pretrained features; Kumar et al. [3] show that full fine-tuning can damage pretrained representations and generalize worse than training only the head.
3. **Cost:** only 3.68 % of the parameters are trained and the backbone features can be computed once and reused. This makes personalization fast and memory-efficient.

4.2.4 Output Activation

The output activation is applied after the last linear layer that produces (x, y). Sigmoid maps the output to $(0, 1)$, which matches the normalized coordinate range. The Gaussian activation $g(z) = \exp(-z^2/2\sigma^2)$ with $\sigma = 1$ also maps to $(0, 1]$ but is not monotonic. ReLU only removes negative values, and "None" leaves the linear output unchanged.

Table 4.7: Output activation. Mean test error over five runs with 98 % CI and mean duration of one complete training run.

Activation	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
Sigmoid†	**28.36**	[28.29, 28.43]	—	0.022
Gaussian	31.29	[28.44, 34.14]	0.008	0.019
ReLU	32.90	[19.69, 46.10]	0.008	0.028
None	29.39	[28.42, 30.36]	0.008	0.032

Sigmoid is the only clear winner of the hyperparameter study (Table 4.7): it has the lowest mean error and the narrowest interval, and *every* Sigmoid run is better than *every* run of the three other activations ($p = 0.008$ in all three comparisons). The value is therefore printed in bold. The other activations are also less stable: one ReLU run ends at 46.98 %, which is worse than the random baseline. A plausible reason is that ReLU outputs can become zero for all inputs ("dead" outputs), from which the gradient cannot recover. Without a bounding activation (None, ReLU) the network can also predict points outside the screen. Sigmoid is selected because it bounds the output to the valid screen range and gave the lowest and most stable error.

Table 4.8: Data set size (train/test). Mean test error over five runs with 98 % CI, mean duration of one complete training run and the constant baseline of the respective test subset.

Train / test	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)	Const. baseline (%)
10 000 / 2000	27.64	[27.55, 27.73]	0.008	0.167	27.62
1000 / 200†	28.39	[28.33, 28.44]	—	0.018	28.46

4.2.5 Data Set Size

The larger configuration has a lower mean error (Table 4.8). However, the two configurations are evaluated on *different* test sets, and each mean is practically equal to the constant baseline of its own test set (27.64 % vs. 27.62 % and 28.39 % vs. 28.46 %). The difference of 0.75 percentage points is therefore caused by the different test data and not by more training data, even though the rank test is significant ($p = 0.008$). The small configuration (1000/200) is kept for the sweeps because it is about ten times faster and corresponds to the data volume of a personalization session.

4.2.6 Split Type

Table 4.9: Split type. Mean test error over five runs with 98 % CI and the constant baseline of the respective test subset.

Split	Mean test error (%)	98 % CI (%)	p vs. †	Constant baseline (%)
`norm_random`	27.72	[27.64, 27.80]	0.008	27.66
`norm_subject`†	28.39	[28.26, 28.53]	—	28.46

The random split gives the lower error (Table 4.9). Again, most of the difference comes from the test subsets: the constant baseline is already 0.80 percentage points lower on the `norm_random` subset, and both models are within 0.07 percentage points of their own baseline. In addition, the random split lets the same persons appear in training and test data, which makes the task easier and the estimate optimistic. The subject-disjoint split `norm_subject` is therefore used as the default: it is the only one that measures generalization to unseen persons, which is the situation before personalization.

4.3 Spatial Error Analysis and Loss Functions

The mean error of Section 4.2 hides *where* on the screen the errors occur. This section first analyses the spatial error distribution (Sections 4.3.1 and 4.3.2). The observed pattern motivated position-weighted loss functions (Section 4.3.3), which are evaluated in Section 4.3.4.

4.3.1 Screen Partition

The normalized screen $[0,1]^2$ is described by a 3×3 reference grid with three regions:

$$\begin{aligned} \text{Centre:} &\quad (0.50,\, 0.50), \\ \text{Edges:} &\quad (0.10,\, 0.50),\ (0.50,\, 0.10),\ (0.90,\, 0.50),\ (0.50,\, 0.90), \\ \text{Corners:} &\quad (0.10,\, 0.10),\ (0.10,\, 0.90),\ (0.90,\, 0.10),\ (0.90,\, 0.90). \end{aligned}$$

A test sample with target $\boldsymbol{p}$ belongs to grid point $\boldsymbol{g}$ if $\|\boldsymbol{p} - \boldsymbol{g}\|_2 \leq r$ with $r = 0.15$. The error of a grid point is the mean Euclidean error $\|\hat{\boldsymbol{p}} - \boldsymbol{p}\|_2$ of its samples (in normalized units, i.e. a fraction of the screen side; the diagonal is $\sqrt{2} \approx 1.41$). The error of a region is the mean over all samples of its grid points. These definitions are used in all figures and tables of this section.

4.3.2 Error Heat Maps

Figure 4.3 uses the 200 test predictions of the default model (`norm_subject`, L1 loss; file `results/heatmap/base_loss1.csv`). For a cursor position $\boldsymbol{c}$ (centre, edge or corner), the mean error $\bar{e}(\boldsymbol{c})$ of all samples within radius r is computed and shown as an isotropic Gaussian

$$H(x, y) = \bar{e}(\boldsymbol{c}) \exp\left(-\frac{\|(x, y) - \boldsymbol{c}\|_2^2}{2\sigma^2}\right), \qquad \sigma = 0.20. \tag{4.3}$$

The colour therefore encodes the *size* of the local mean error, and the position of the bright spot shows *where* it was measured. All three panels use the same colour scale (0 to the largest value), so the colours can be compared directly.

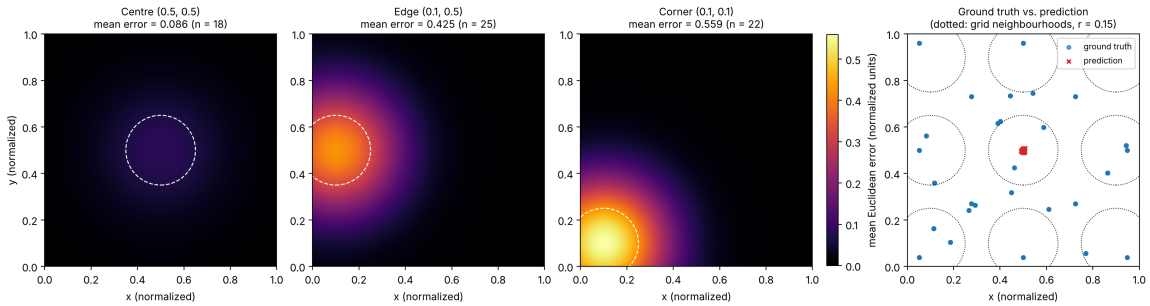


Figure 4.3: Spatial error of the default model (L1 loss, `norm_subject`, 200 test samples). Panels 1–3: Gaussian error heat maps ($r = 0.15$, $\sigma = 0.20$, common colour scale) at the centre $(0.5, 0.5)$, the left edge $(0.1, 0.5)$ and the lower left corner $(0.1, 0.1)$. The error grows from the centre (0.086) to the edge (0.425) and the corner (0.559). Panel 4: targets (blue) and predictions (red); all predictions lie close to the screen centre.

The error increases monotonically from the inside to the outside: the mean error is 0.086 in the centre region ($n = 18$), 0.432 at the edges ($n = 62$) and 0.586 in the corners ($n = 42$).

Edge errors are about five times and corner errors about seven times larger than centre errors.

The fourth panel explains this pattern. All predictions lie in a very small area around $(0.5, 0.5)$: their standard deviation is about 0.004 in both coordinates, while the targets have a standard deviation of about 0.28. The model therefore predicts almost the same point for every image, and its error is simply the distance of the target to the screen centre. A constant centre predictor gives practically the same values (centre 0.085, edges 0.432, corners 0.587; overall $E_{\text{diag}} = 28.45\,\%$ versus $28.47\,\%$ for the model).

4.3.3 Loss Functions

The high edge and corner errors motivated the idea to give these regions more weight in the loss. Let $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ be the prediction and $\boldsymbol{p}_i = (x_i, y_i)$ the target of sample i in a batch of size N.

L1 loss (`nn.L1Loss`, "standard").

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} (|\hat{x}_i - x_i| + |\hat{y}_i - y_i|). \tag{4.4}$$

The gradient has a constant magnitude independent of the error size. The loss is therefore robust against outliers (for example wrong labels during target transitions), but it treats all screen positions equally.

Smooth L1 loss (`nn.SmoothL1Loss`, $\beta = 1$). For a residual u:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{otherwise.} \end{cases} \tag{4.5}$$

It is quadratic for small and linear for large residuals. Because all residuals in $[0, 1]^2$ are smaller than $\beta = 1$, it behaves here like a scaled squared error: the gradient becomes small when the error is small.

Sensitive L1 loss (position-weighted, `sensitive`). Each sample is weighted by its L_1 distance from the screen centre:

$$d_i = |x_i - 0.5| + |y_i - 0.5| \in [0, 1], \qquad w_i = 1 + d_i \in [1, 2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

A corner sample has twice the weight of a centre sample, an edge sample about 1.4 times. This increases the gradient for outer targets and should pull the predictions away from the centre. (The earlier draft defined the weight as $w = 1/d$; this does not match the implementation in `model.py` and would be infinite at the centre.)

The "improved sensitive loss" in Table 4.11 was a further variant from an exploratory notebook. Its exact formula is not contained in the current code, so it is reported only for completeness.

4.3.4 Results of the Loss Comparison

Table 4.10 combines the global comparison from the five-run sweep with the spatial errors of the regions defined in Section 4.3.1. It replaces the two separate tables of the earlier draft (global loss table and 3×3 grid table).

Table 4.10: Comparison of L1 and Sensitive L1 loss. Left: mean test error E_{diag} over five runs (`exp.py`, 98 % CI) and mean run time. Right: mean Euclidean error per screen region (normalized units; one earlier run each, 200 test samples, $r = 0.15$; n = number of samples). Last row: constant screen-centre predictor on the same 200 test samples.

	Global (5 runs)			Spatial error (1 run)		
Loss	E_{diag} (%)	98 % CI (%)	Run time (min)	Centre ($n = 18$)	Edges ($n = 62$)	Corners ($n = 42$)
L1 (standard)[†]	**28.32**	[28.23, 28.42]	0.011	0.086	0.432	0.586
Sensitive L1	28.48	[28.19, 28.78]	0.021	0.090	0.431	0.585
Constant centre	28.45[a]	—	—	0.085	0.432	0.587

[a] Evaluated on the same 200 test samples as the spatial analysis.

Table 4.11: Auxiliary experiment: spatial error of four loss functions after 15 epochs on a training set with 52 corner samples (two runs, A and B, values as logged). These numbers come from an exploratory notebook; the computation of the region values of this run is not contained in the current code, so they are not directly comparable with Table 4.10.

	Total (%)		Centre (%)		Edges (%)		Corners (%)	
Loss	A	B	A	B	A	B	A	B
L1 (`nn.L1Loss`)	24.54	24.63	17.01	16.74	28.28	28.98	34.93	35.07
Smooth L1 (`nn.SmoothL1Loss`)	23.94	24.91	15.40	16.41	29.78	30.59	34.13	35.17
Sensitive L1	25.57	25.55	19.11	18.61	29.87	29.93	33.41	34.20
Improved sensitive L1	25.53	27.75	19.20	21.86	29.17	35.03	33.77	31.58

Observations.

- Globally, L1 is slightly but significantly better than the sensitive loss: 28.32 % versus 28.48 % (Mann–Whitney $p = 0.032$, Table 4.10); the L1 value is therefore printed in bold. The sensitive loss also varies more between runs. Both values lie close to the constant baseline (28.46 %).
- Spatially, the position weighting changes the region errors by less than 0.005 normalized units. The edge and corner errors stay about five to seven times higher than the centre error.
- Both models have the same spatial profile as a constant centre predictor. The weighting increases the gradient for outer samples, but this is not enough to make the model leave the centre solution.
- In the auxiliary experiment (Table 4.11) the sensitive variants slightly reduce the corner error (Sensitive L1 by 0.9–1.5, the improved variant by 1.2–3.5 percentage points), but they increase the centre error by 1.9–5.1 percentage points and the total error in all runs. The two runs differ considerably, so these trends are not reliable.

Conclusion. The spatial analysis showed *what* the problem is: the errors are concentrated at the edges and in the corners. The position-weighted loss was the attempt to reduce these errors. It did not improve the corners and was globally even slightly but significantly worse than L1 ($p = 0.032$); in the auxiliary experiment it increased the centre error. The standard L1 loss is therefore kept. It gave the lower error, it is simpler, it has no additional hyperparameter, and its constant gradient magnitude makes it robust against label noise in the personal recordings. The analysis also shows *why* the weighting could not help: the large outer errors are not caused by a wrong weighting of screen regions but by the fact that the model barely leaves the screen centre (Section 4.7).

4.4 Personalization Sessions

4.4.1 Setup

In each personal session the head of a GazeCapture base model is trained again on the frames of one recording (FC only, Sigmoid, L1, $\eta = 10^{-4}$, weight decay 10^{-5}, batch size 32, AdamW). Unless stated otherwise each session is split randomly into 70 % training, 15 % validation and 15 % test frames (seed 42). Early stopping (patience 3) uses the validation error, and the test error is reported at the epoch with the lowest validation error. If the best epoch is 0, the personalization did not improve the validation error and the reported test error is that of the unchanged base model.

All results of this and the following sections were produced with the script `personal_exp.py` (notebook `personal_compare.ipynb`) and the corrected metric (Equation (4.1)). All random seeds are fixed, and the backbone, including its BatchNorm statistics, stays completely frozen.

Feature cache. Because the backbone is frozen, it maps an image to the same 512 features in every epoch. These features are computed once per session and base model (evaluation transform, no augmentation) and stored; afterwards only the regression head is trained. This does not change the trained model, but it makes a run take seconds on a CPU. The only difference to the earlier notebook runs is that no image augmentation is applied during personalization.

Base models (E0). The two base models are trained with the selected configuration of Section 4.2 (FC only, Sigmoid, L1, $\eta = 10^{-4}$, batch 32, 1000/200 random frames, seed 0, validation-based early stopping) on `norm_subject` and `norm_random`. Of five seeds the run with the lowest *validation* error is kept (`train_base.py`, `models/base/base_<split>.pt` with a JSON file that documents the configuration and all seeds). For `norm_subject` seed 0 was selected (validation 27.37 %, test 28.70 %; constant baseline of the same test data 28.46 %); for `norm_random` seed 3 (validation 26.74 %, test 27.81 %; constant baseline 27.66 %). The five seeds differ by less than 0.4 percentage points, so the choice of the seed is not critical. As in Section 4.7, both base models are at the level of the constant baseline.

Of the 27 recorded sessions, 18 are used: Gen 1 (01–04), Gen 2 (05–09) and Gen 4 (19–27). Gen 3 (10–18) was discarded because it showed no learning effect. Table 4.12 lists the recording conditions.

4.4.2 Results of All Sessions (E1)

Table 4.13 shows every session with both base models.

Personalization works. Over all 18 sessions the test error falls from 30.32 % (constant baseline) to 19.79 % with the `norm_subject` base model and to 19.59 % with the `norm_random` base model, a reduction of 35 % and 36 % (Wilcoxon signed-rank test $p = 1.5 \cdot 10^{-5}$ for both; paired t-test $p < 10^{-6}$). The unchanged base models are not better than the constant baseline (30.57 % and 30.51 %; Wilcoxon $p = 0.25$ and $p = 0.35$), so the whole gain comes from the few minutes of personal data. The best session, S02 (Gen 1, bright room, no glasses), reaches 10.5 %. The new runs confirm the earlier notebook runs (mean 19.85 % over the same sessions with the `norm_subject` base model).

Table 4.12: Recording conditions of the 18 sessions. Gen 1: continuous target movement without jumps; Gen 2: target with random jumps; Gen 4: 5-s segments (0.5 s WAIT at a new start point, 4.5 s MOVE), 90 % of the start points in the screen corners. Duration and operating system were only varied in Gen 4 (Gen 1/2: Ubuntu, duration not logged).

Sessions	Generation	Glasses	Light / background
01, 03, 04	Gen 1	with	01, 04: bright; 03: dark
02	Gen 1	without	bright
05, 07, 08	Gen 2	with	05: bright; 07: dark; 08: very dark
06, 09	Gen 2	without	06: bright; 09: very dark
Gen 4	(duration / OS)		
19, 20	Gen 4	with	clear; 120 s / 90 s; Ubuntu
21, 22	Gen 4	without	clear; 120 s / 90 s; Ubuntu
23 / 24	Gen 4	with / without	noisy; 120 s; Ubuntu
25 / 26	Gen 4	with / without	noisy; 120 s; Windows
27	Gen 4	with	noisy; 90 s; Windows

Table 4.13: Personalization of the 18 sessions with two base models (original labels, random 70/15/15 split, seed 42). Base: test error of the unchanged base model (epoch 0); Val.: best validation error; Test: test error at the best validation epoch; Ep.: best epoch; t: duration of one run in seconds (Section 4.4.3); Const.: constant baseline (mean of the training targets, evaluated on the same test frames). Errors in % (Equation (4.1)). Group rows: mean and 98 % CI over the sessions of the group.

		Base model `norm_subject`					Base model `norm_random`					
Session	Gen.	Base	Val.	Test	Ep.	t (s)	Base	Val.	Test	Ep.	t (s)	Const.
01	1	29.19	14.82	15.83	22	2.3	29.49	11.70	12.97	47	4.2	29.45
02	1	28.48	10.08	10.48	30	2.2	28.42	10.04	10.43	44	3.2	28.49
03	1	29.61	15.26	17.20	33	2.5	30.13	15.41	17.35	36	2.6	29.76
04	1	29.98	16.88	17.64	17	2.1	29.38	15.23	16.16	31	2.7	29.17
05	2	32.92	25.43	17.64	26	1.1	32.49	31.93	32.77	2	0.2	32.72
06	2	29.29	11.31	16.67	54	1.8	28.89	12.09	16.55	55	2.0	27.02
07	2	32.18	24.84	30.99	16	0.7	32.43	25.02	31.62	18	0.6	32.28
08	2	25.82	23.65	23.30	24	0.8	26.32	23.38	22.31	23	0.6	27.00
09	2	26.83	23.55	28.64	4	0.2	26.21	15.19	16.87	47	1.9	28.17
19	4	30.42	13.38	17.76	33	2.8	30.04	12.13	16.85	47	4.4	30.19
20	4	33.14	16.06	14.64	23	1.9	33.01	14.58	12.93	35	2.7	33.49
21	4	30.18	15.03	12.51	39	4.6	30.46	15.80	13.20	23	2.6	29.97
22	4	31.17	13.54	13.02	22	2.2	30.87	12.53	12.32	37	3.1	30.32
23	4	33.15	23.48	23.65	10	1.4	32.59	24.02	24.05	10	2.0	32.57
24	4	33.09	28.77	26.97	7	0.9	32.84	29.53	27.42	6	1.0	33.01
25	4	32.19	26.98	24.61	19	2.4	33.00	26.74	24.13	26	2.8	31.54
26	4	30.60	21.93	18.61	20	2.3	30.39	21.40	17.97	37	4.5	29.48
27	4	31.98	26.05	26.06	19	1.5	32.29	26.27	26.68	19	1.7	31.19
Gen 1 (4)		29.32	14.26	15.29	25.5	2.3	29.35	13.10	14.23	39.5	3.2	29.22
		[27.86, 30.77]	[7.62, 20.89]	[7.80, 22.77]		[1.8, 2.7]	[27.75, 30.96]	[7.07, 19.12]	[7.10, 21.35]		[1.5, 4.8]	[27.99, 30.45]
Gen 2 (5)		29.41	21.76	23.45	24.8	0.9	29.27	21.52	24.02	29.0	1.0	29.44
		[24.14, 34.68]	[11.88, 31.63]	[12.74, 34.16]		[0.0, 1.9][a]	[24.06, 34.47]	[8.20, 34.85]	[10.93, 37.12]		[0.0, 2.5][a]	[24.68, 34.20]
Gen 4 (9)		31.77	20.58	19.76	21.3	2.2	31.72	20.33	19.51	26.7	2.8	31.31
		[30.60, 32.94]	[14.66, 26.50]	[14.26, 25.26]		[1.2, 3.3]	[30.51, 32.93]	[13.88, 26.79]	[13.59, 25.42]		[1.7, 3.9]	[29.91, 32.70]
All (18)		**30.57**	**19.50**	**19.79**	**23.2**	**1.9**	**30.51**	**19.05**	**19.59**	**30.2**	**2.4**	**30.32**
		[29.27, 31.87]	[15.87, 23.13]	[16.18, 23.40]		[1.3, 2.5]	[29.21, 31.82]	[14.86, 23.25]	[15.47, 23.71]		[1.6, 3.2]	[29.12, 31.53]

[a] Lower limit of the t interval below zero (few sessions, large spread), truncated at 0.

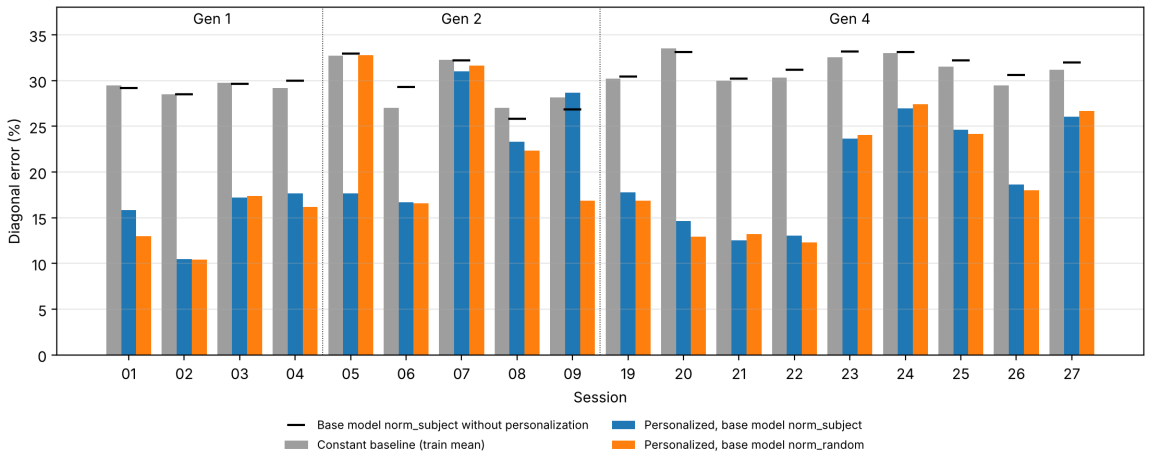


Figure 4.4: Test error of the 18 sessions after personalization with the two base models, compared with the constant baseline and with the unchanged base model (`norm_subject`) on the same test frames (data of Table 4.13).

Groups. The reduction compared with the constant baseline is 48 % / 51 % in Gen 1, 20 % / 18 % in Gen 2 and 37 % / 38 % in Gen 4 (`norm_subject` / `norm_random`). In Gen 4 the improvement is significant (Wilcoxon $p = 0.004$); for Gen 1 ($n = 4$) and Gen 2 ($n = 5$) the smallest possible p-value of the test is 0.125 and 0.0625, so these groups are too small for a significant result although the effect in Gen 1 is large. The differences between the generations are not significant (Kruskal–Wallis $p = 0.13$ and $p = 0.12$) and must not be read as an effect of the recording protocol alone (Section 4.6).

When personalization fails. In two runs the personalized model is not better than the constant baseline: S09 with `norm_subject` (best epoch 4, 28.64 % vs. 28.17 %) and S05 with `norm_random` (best epoch 2, 32.77 % vs. 32.72 %). In both cases the validation error stopped improving very early. With the other base model the same sessions reach 16.87 % (S09) and 17.64 % (S05). The failure is therefore not a property of the session but of a single run: with only 20–30 validation frames (Gen 2), early stopping can end the training after a few epochs. S07 (dark room) improves only slightly with both base models (31.0 % / 31.6 % vs. 32.3 %); good illumination remains a precondition for webcam personalization.

Base model. Both base models give the same result (19.79 % vs. 19.59 %; Wilcoxon $p = 0.20$). This is consistent with the main finding that the base model mainly provides an initialization and that the personal data determine the result. The `norm_subject` base model is used for the following experiments, because it was trained without information about the test persons.

Validation and test error. On average the validation error (19.50 %) is close to the test error (19.79 %). The largest gaps (5–8 percentage points, S05–S07, S09) occur in the short Gen 2 recordings, where the validation and test sets contain only about 25 frames each.

4.4.3 Training Time of a Session

The time t in Table 4.13 is the wall-clock duration of *one complete personalization run* of a session on the cached features: the evaluation of the unchanged model (epoch 0), all training epochs until early stopping, and the evaluation of the training, validation and test data after every epoch. It is neither a time per epoch nor a mean over several runs. It does not include the one-time extraction of the backbone features of the session, which is shared by all runs of the session and base model. A run takes on average 1.9 s (`norm_subject`) and 2.4 s (`norm_random`) on a CPU; it grows with the number of frames and the number of epochs (short Gen 2 recordings: 0.9 s). The earlier notebook runs, which passed every image through the backbone in every epoch, needed 1–26 minutes per session. In contrast, the "Run time" of the hyperparameter tables (Section 4.1.4) is the mean over five repeated runs of a GazeCapture configuration.

4.4.4 Label Cleaning (Gen 4, E1b)

In Gen 4 every segment starts with a 0.5 s WAIT phase in which the target appears at a new position and the user still has to find it. These frames (10.0–10.5 % of the labels) can have a wrong gaze label. They are logged exactly, so cleaned label files without the WAIT frames can be written (`session_design.py -write-clean`).

A naive comparison is not fair: the cleaned model is tested without WAIT frames, while the original model is tested on all frames, including the WAIT frames that are hard to predict. Therefore the original model is also evaluated on the MOVE frames of its test split only (same model, WAIT frames removed from the test set; Table 4.14).

Result. The WAIT frames are predicted very badly (37.7 %, above the constant baseline): the gaze is not yet on the target, so their labels are wrong. Removing them from the *test* set alone lowers the error from 19.76 % to 16.72 %. Removing them also from the *training* data lowers it further to 15.88 %, a mean gain of 0.84 percentage points that is not significant (Wilcoxon $p = 0.43$). The gain comes from two sessions with a noisy background (S23, S25, about -3.7 points); in the clear sessions the difference is below 0.6 points. Conclusion: the WAIT frames should be removed at least from the evaluation, because they measure the label error and not the model error; removing them from training is cheap and helps in difficult sessions, but the effect is small.

Table 4.14: Label cleaning in Gen 4 (base model `norm_subject`, random split, seed 42). Original: trained on all labels; test error on all test frames, on the MOVE frames only and on the WAIT frames only (n_W = number of WAIT test frames). Clean: trained and tested without WAIT frames. Δ = Clean − Original (MOVE).

	Original labels				Clean labels		
Session	All	MOVE	WAIT	n_W	Test	Const.	Δ
19	17.76	13.28	41.12	8	13.06	29.07	−0.22
20	14.64	12.24	28.38	8	12.77	30.71	+0.53
21	12.51	9.77	27.59	10	9.79	29.23	+0.01
22	13.02	10.21	27.64	8	9.96	31.71	−0.25
23	23.65	20.95	41.84	10	17.18	30.92	−3.77
24	26.97	24.96	41.93	10	23.99	31.64	−0.97
25	24.61	21.63	44.36	10	17.94	30.49	−3.70
26	18.61	14.34	41.72	10	14.58	30.20	+0.24
27	26.06	23.10	44.80	8	23.69	30.59	+0.60
Mean	19.76	16.72	37.71	9.1	15.88	30.51	−0.84

Run-to-run variability. The invalid earlier comparison consisted of two notebook runs of the same configuration on identical data (Gen 4, random seeds not fixed). They differed on average by 1.9 and at most by 5.5 percentage points. The new runs with three fixed seeds (Table 4.15) show a standard deviation of up to 3.1 percentage points between seeds. Differences between *single* runs of this size are therefore not interpretable; this also explains the S05/S09 cases above.

4.4.5 Recording Conditions (Gen 4)

Gen 4 allows comparisons in which only one factor changes. Each value is the difference of two single sessions (test error of Table 4.13, `norm_subject` / `norm_random`, percentage points; positive = the first condition is worse):

- **Glasses** (with − without): +5.2/ + 3.6 (S19–S21), +1.6/ + 0.6 (S20–S22), −3.3/ − 3.4 (S23–S24), +6.0/ + 6.2 (S25–S26);
- **Duration** (90 s − 120 s): −3.1/ − 3.9 (S20–S19), +0.5/ − 0.9 (S22–S21);
- **Background** (noisy − clear): +5.9/ + 7.2 (S23–S19), +14.5/ + 14.2 (S24–S21);
- **Operating system** (Windows − Ubuntu): +1.0/ + 0.1 (S25–S23), −8.4/ − 9.4 (S26–S24).

The only consistent and large effect is the **background**: a noisy background increases the error by 6–15 percentage points with both base models. Glasses tend to increase the error (three of four pairs), the recording length (90 vs. 120 s) shows no clear effect, and

the operating system comparison is inconsistent. Each condition is represented by single sessions, so these values are descriptive and not statistically tested.

4.5 Data Loading Strategies (E2)

The random split of Section 4.4 has a weakness: neighbouring frames of a video are almost identical (6 labels per second), so the test set contains near-duplicates of training frames. This is a form of data leakage that makes the random split optimistic. The following comparison measures how large this effect is.

Sessions. Sessions 21 and 22 are used. They are selected by a fixed rule: they are the only two sessions that differ in a single factor (duration, 120 s vs. 90 s) and were otherwise recorded under the best conditions (Gen 4, no glasses, clear background, Ubuntu, same day). This makes them suitable for the two-video transfer, and both have enough frames (720 and 540) for temporal blocks.

Strategies.

Random sampling (`-split random`): 70 % / 15 % / 15 % of the frames are drawn at random from the session (reference; contains near-duplicates).

Sequential sampling (`-split sequential`): the frames are kept in recording order; the first 70 % are used for training, the next 15 % for validation and the last 15 % for testing. One second (6 labels) at the start of the validation and of the test block is dropped, so that no neighbouring frames lie on both sides of a block border. This corresponds to the real use case: the user calibrates first and uses the tracker afterwards.

Two-video training (`-split two_videos`): 80 % of the frames of the first session are used for training and the remaining 20 % (random) for validation; the second session is used *only* for testing (all frames). Both directions are run (21→22 and 22→21). This measures whether a calibration transfers to a new recording of the same person.

Fair comparison. All strategies use the same base model (`norm_subject`), the same training settings, early stopping on their own validation data and the same three seeds (42, 43, 44). For every run the constant baseline (mean of the *training* targets, evaluated on the test frames of that run) and the error of the unchanged base model on the same test frames are reported, so that the different test sets can be compared. No test frame is used for training or for model selection.

Table 4.15: Data loading strategies (sessions 21 and 22, base model `norm_subject`, original labels). Mean over three seeds; SD: standard deviation of the test error over the seeds; Base: unchanged base model; Red.: reduction of the test error relative to the constant baseline; n: training / validation / test frames; t: duration of one run (s).

Strategy	Train → test	Test (%)	SD	Val. (%)	Base (%)	Const. (%)	Red. (%)	n	t (s)
Random	21 → 21	14.40	3.13	14.15	31.52	31.25	53.9	503 / 108 / 109	3.9
	22 → 22	13.30	0.38	15.98	30.97	30.68	56.7	378 / 81 / 81	3.1
Sequential	21 → 21	14.47	1.22	19.09	27.62	27.43	47.2	503 / 102 / 103	2.0
	22 → 22	16.63	0.14	14.41	28.83	29.91	44.4	378 / 75 / 75	2.4
Two videos	21 → 22	16.79	0.13	15.67	31.05	32.17	47.8	576 / 144 / 540	5.0
	22 → 21	17.70	0.42	14.33	32.03	33.13	46.6	432 / 108 / 720	3.2
Random (mean)		13.85			31.25	30.97	55.3		
Sequential (mean)		15.55			28.22	28.67	45.8		
Two videos (mean)		17.24			31.54	32.65	47.2		

Result. The expected order is confirmed: random (13.85 %) < sequential (15.55 %) < two videos (17.24 %). With 6 runs per strategy (two sessions × three seeds) only the difference between sequential and two-video training is significant (Mann–Whitney U, $p = 0.009$); random vs. sequential ($p = 0.09$) and random vs. two videos ($p = 0.06$) are not. Because the constant baselines differ between the test sets, the relative reduction is the fairer measure: 55 % for the random split, but only 46 %–47 % for the realistic strategies. The random split thus overestimates the benefit of personalization by roughly 8–9 percentage points of reduction.

Interpretation.

- The random split is optimistic, but on these two good sessions only moderately (1.7 percentage points). The effect is much larger on difficult sessions (Section 4.6.2).
- A calibration transfers to a second recording of the same person under the same conditions: the two-video model reaches 17 %–18 % on a video it has never seen, with 47 % reduction. The transfer costs about 3 percentage points compared with the random split.
- The two-video results vary least between seeds (SD below 0.5), as the test set is large (540–720 frames). The random split has the largest spread (SD 3.1 in S21), because its 81–109 test frames depend on the seed.
- The validation error of the sequential split does not predict its test error well (S21: 19.1 % validation vs. 14.5 % test), because the validation and test blocks cover different screen regions.

For the use case of an eye tracker, which is calibrated once and then used, the sequential or two-video values are the realistic estimate.

4.6 Personalization Session Design

The session design determines which screen positions are covered by the calibration data. Because the errors are largest at the edges and in the corners (Section 4.3), the design aims to cover these regions well. Three designs were recorded:

- **Standard video** (Gen 1, `base_Code.py`): the target moves continuously over the screen, without jumps;
- **Random jumping points** (Gen 2): the target jumps every 1.4–2.7 s to a uniformly drawn new start point;
- **Corner-weighted jumping points** (Gen 4, `base_Code_rand.py`): 5-s segments with 0.5 s WAIT at a new start point and 4.5 s MOVE; 90 % of the start points lie in the four corner regions (each 10 % × 10 % of the screen).

4.6.1 Spatial Coverage and Transition Samples

Table 4.16 is computed directly from the label and position files of the 18 sessions (`session_design.py`); no training is needed. The region shares use the screen partition of Section 4.3.1 ($r = 0.15$). Transition samples are labels recorded while the gaze is still moving to a new target: in Gen 4 the logged WAIT phase, in Gen 2 the labels within 0.5 s after a jump (estimated, no phase logging), in Gen 1 none.

Table 4.16: Spatial coverage and transition samples per design (mean over the sessions of each generation). Cells: share of the 100 cells of a 10 × 10 screen grid that contain at least one label; Dist.: mean distance of the targets from the screen centre (normalized units).

Design	Labels	Length (s)	Jumps	Transition (%)	Share of labels (%)			Cells (%)	Dist.
					Centre	Edges	Corners		
Standard (Gen 1, 4 sess.)	493	82	0	0	7.1	25.0	23.0	97	0.383
Random jumps (Gen 2, 5 sess.)	179	30	16	28.5[a]	6.3	23.0	26.3	57	0.384
Corner-weighted (Gen 4, 9 sess.)	644	107	20	10.1	6.1	19.5	36.1	91	0.420

[a] Estimated: labels within 0.5 s after a jump.

Observations.

- **Corner weighting works as intended:** Gen 4 has 36 % of its labels in the corner neighbourhoods compared with 23 % (Gen 1) and 26 % (Gen 2), and the largest mean distance from the centre.
- **Continuous movement covers the screen most evenly:** Gen 1 reaches 97 % of the grid cells. The short Gen 2 recordings (30 s) cover only 57 %.

- **Transition samples:** with jumps every 1.4–2.7 seconds and no phase logging, up to a third of the Gen 2 labels fall into a transition; Gen 4 has 10 %, which are logged exactly and can be removed (Section 4.4.4).

4.6.2 Personalization Performance (E3)

The random split favours designs with slow, continuous movement, because neighbouring, almost identical frames end up in training and test data. Therefore all 18 sessions were also run with the sequential split (Section 4.5; base model `norm_subject`, seed 42). Table 4.17 compares both splits per design, and Table 4.18 shows the errors per screen region, computed from the stored test predictions.

Table 4.17: Test error per design with the random and the sequential split (mean over the sessions; base model `norm_subject`). Const.: constant baseline of the respective test frames; Ep. 0: number of sessions in which the best epoch was 0 (no improvement over the base model); n_{test}: mean number of test frames.

	Random split		Sequential split			
Design	Test	Const.	Test	Const.	Ep. 0	n_{test}
Standard (Gen 1)	15.29	29.22	21.52	28.70	1 / 4	69
Random jumps (Gen 2)	23.45	29.44	32.80	34.39	2 / 5	22
Corner-weighted (Gen 4)	19.76	31.31	28.07	31.82	5 / 9	91
All (18)	19.79	30.32	27.93	31.84[a]	8 / 18	72

[a] Mean of the session values of the sequential split.

Table 4.18: Test error per screen region (Section 4.3.1, $r = 0.15$), mean over the sessions of a design; in brackets the total number of test frames in the region. Base model `norm_subject`.

	Random split				Sequential split			
Design	All	Centre	Edges	Corners	All	Centre	Edges	Corners
Standard (Gen 1)	15.3	17.7 (20)	13.5 (67)	16.7 (83)	21.5	11.9 (25)	20.9 (64)	28.9 (52)
Random jumps (Gen 2)	23.4	13.7 (12)	21.6 (34)	26.1 (42)	32.8	6.6 (2)[a]	29.8 (15)	38.6 (45)
Corner-weighted (Gen 4)	19.8	11.5 (45)	14.9 (175)	26.5 (299)	28.1	9.0 (60)	23.9 (136)	37.1 (289)
All (18)	19.8	13.5 (77)	16.4 (276)	24.2 (424)	27.9	9.7 (87)	24.3 (215)	36.1 (386)

[a] Only one session with two centre frames; not interpretable.

Sequential split. Without near-duplicates the error rises in every design, on average from 19.8 % to 27.9 % (Wilcoxon $p = 4 \cdot 10^{-5}$). The personalized model is still better than the constant baseline (Wilcoxon $p = 0.002$), but the reduction shrinks from 35 % to about 12 %. In 8 of 18 sessions (02, 05, 08, 23–27) the best epoch is 0: training on the first 70 % of the recording does not improve the error on the validation block, so the reported result is the

unchanged base model. This happens mainly in Gen 4 with a noisy background (23–27) and in the short Gen 2 recordings, whose validation and test blocks contain only 17–27 frames. The random split of Section 4.4 therefore strongly overestimates the benefit of personalization for difficult sessions.

Designs. With both splits the standard video (Gen 1) has the lowest error and the random jumps (Gen 2) the highest; the differences are not significant with the sequential split either (Kruskal–Wallis $p = 0.06$). The ranking can be explained by the coverage (Table 4.16): Gen 1 covers 97 % of the screen in every part of the recording, while in the jump designs a temporal block covers only the targets shown in that time. The short Gen 2 recordings suffer most. The designs are still confounded with illumination, background and glasses, so the ranking is a tendency, not a proof.

Regions. The corners are the hardest region in every design and with both splits (24 % random, 36 % sequential), the centre the easiest (10–14 %). The corner-weighted design does *not* reduce the corner error: Gen 4 has 26.5 % / 37.1 % in the corners, compared with 16.7 % / 28.9 % for the standard video. More corner labels alone do not help when the gaze in the corners is harder to estimate (head rotation, eyes partly closed, camera geometry) and when the corner labels are recorded under worse conditions (Gen 4 sessions 23–27). The extreme corners are also the points where the sigmoid output must be close to 0 or 1.

Conclusion for the design. For a short calibration, a continuous target movement that covers the whole screen gives the best result with the existing data. Jumps produce transition samples (Gen 2: up to 28 % of the labels) and, if used, must be logged so that these frames can be removed. Corner weighting raises the share of corner labels as intended, but does not lower the corner error.

4.7 Discussion

Main finding. On GazeCapture, all configurations of the hyperparameter study reach practically the error of a constant predictor on the same test data: the best FC-only configurations lie between 28.32 % and 28.39 %, the constant baseline is 28.46 %. The spatial analysis confirms that the predictions are concentrated at the screen centre. Consequently the hyperparameter study separates the configurations mainly by *stability*: options that make training less stable (Gaussian, ReLU, no activation, unfreezing backbone layers, the sensitive loss) are worse, but no option clearly leaves the centre solution. The choices in Section 4.2 are therefore based on stability, convergence, cost and methodological arguments, as stated in each subsection. Single notebook runs on `norm_random` (Figures 4.1b and 4.2)

reached about 23.6 % versus 26.8 % for the constant baseline, so a small learning effect exists, but it is not visible in the five-run means. The base models of the personalization confirm this: their error on the personal sessions (30.6 %) equals the constant baseline.

Possible reasons. The new protocol already excludes two explanations that were plausible for the earlier runs: the training subset now contains 179 subjects instead of five, and the test set is no longer used for model selection; the collapse remains. The most likely remaining causes are: (i) full camera frames are used, in which the eyes cover only a small part of the image, so the frozen ImageNet features carry little gaze information; (ii) 1000 frames from 179 subjects are only about six frames per person, too few to learn a person-independent mapping; unfreezing more layers with so few data makes training unstable instead of better (Section 4.2.3). Face or eye crops as input and larger training sets are the next steps to test.

Personalization. With the same frozen backbone, a head trained on a few hundred frames of one person reduces the error by about one third compared with the constant baseline (random split), in about two seconds on a CPU. The choice of the base model does not matter. This supports the central idea of this work, data-efficient personalization of a pretrained model. Two limitations are important: (i) the random split is optimistic; with a temporal split the reduction falls to about 12 % on average and to zero in difficult sessions, whereas good sessions keep most of the gain and a calibration even transfers to a second recording (47 % reduction, Section 4.5); (ii) the recording conditions (background, illumination, glasses) influence the result more than the session design.

Consequences for the code. The evaluation metric was corrected, baselines are computed on the same subset as the model, random subsets (`-sample-seed`) and a subject-disjoint validation set (`-val-size`) were added to `exp.py`, and full fine-tuning (`fc_resnet`) can be included with `-include-resnet`. The personalization runs are reproducible (fixed seeds, frozen BatchNorm, feature cache, stored predictions, `personal_exp.py`).

4.8 Summary of Experimental Results

Table 4.19 summarizes all experiments. “Significant” means $p < 0.05$ in the test given in the respective section. All errors are diagonal errors (Equation (4.1)) in %. MAE and MSE were not logged in these experiments; they are therefore not reported, and the diagonal error (a scaled Euclidean RMSE) is the only accuracy measure that is available and comparable for all experiments.

Table 4.19: Summary of the experimental results and the selected configuration. GazeCapture: `norm_subject`, 1000/200 random frames, five runs, constant baseline 28.46 %; time = mean duration of one training run in minutes (head on cached features). Personalization: mean over the sessions, time = one session run in seconds (Section 4.4.3).

Experiment	Selected	Error (%)	Time	Result and reason
Baselines	—	28.46	—	constant baseline on the same test data; random baseline 40.36 (Section 4.1.6)
Target representation	normalized $[0, 1]^2$	—	—	device-independent; metric well-defined (Section 4.1.5)
Batch size	32	28.39	0.019 min	no significant difference (Section 4.2.1)
Learning rate	10^{-4}	28.37	0.020 min	no significant difference; best convergence in the curves (Section 4.2.2)
Trainable layers	FC only	28.36	0.015 min	lowest and most stable error; FC + Layer 4: 9.2 min, full: 15.4 min (Section 4.2.3)
Activation	Sigmoid	**28.36**	0.022 min	**significantly** better than Gaussian, ReLU, None (Section 4.2.4)
Loss function	L1	**28.32**	0.011 min	**significantly** better than the sensitive loss (Section 4.3.4)
Data set size	1000/200	28.39	0.018 min	larger set better only because of a different test set (Section 4.2.5)
Data split	`norm_subject`	28.39	0.020 min	only split that measures generalization to unseen persons (Section 4.2.6)
Personalization (E1)	FC only, 18 sessions	**19.79**	1.9 s	**significantly** better than the constant baseline (30.32) and the base model (30.57), −35 % (Section 4.4.2)
Base model (E0/E1)	`norm_subject`	19.79	1.9 s	same as `norm_random` (19.59, 2.4 s; $p = 0.20$); chosen because it has no information about the test persons
Label cleaning (E1b)	remove WAIT frames	15.88	—	vs. 16.72 on the same MOVE test frames; −0.84, not significant; WAIT frames themselves 37.7 (Section 4.4.4)
Data loading (E2)	sequential / two videos	15.55 / 17.24	2–5 s	random 13.85 is optimistic; two videos significantly worse than sequential; calibration transfers to a new video (−47 %) (Section 4.5)
Session design (E3)	standard video	15.29 / 21.52	—	random / sequential split; random jumps 23.45 / 32.80, corner-weighted 19.76 / 28.07; not significant and confounded; corner weighting does not lower the corner error (Section 4.6.2)

References

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.